

We have a semantics and implementations

When Concurrency Matters: Behaviour-Oriented Concurrency

[LUKE CHEESEMAN](#), Imperial College London, UK

[MATTHEW J. PARKINSON](#), Microsoft Azure Research, UK

[SYLVAN CLEBSCH](#), Microsoft Azure Research, UK

[MARIOS KOGIAS](#), Imperial College London, UK Microsoft Research, UK

[SOPHIA DROSSOPOULOU](#), Imperial College London, UK

[DAVID CHISNALL](#), Microsoft, UK

[TOBIAS WRIGSTAD](#), Uppsala University, Sweden

[PAUL LIÉTAR](#), Imperial College London, UK

We have a semantics and implementations

When Concurrency Matters: Behaviour-Oriented Concurrency

LUKE CHEESEMAN, Imperial College London, UK

MATTHEW J. PARKINSON, Microsoft Azure Research, UK

SYLVAN CLEBSCH, Microsoft Azure Research, UK

MARIOS KOGIAS, Imperial College London, UK Microsoft Research, UK

SOPHIA DROSSOPOULOU, Imperial College London, UK

DAVID CHISNALL, Microsoft, UK

TOBIAS WRIGSTAD, Uppsala University, Sweden

PAUL LIÉTAR, Imperial College London, UK

$$\begin{array}{c} \text{STEP} \frac{E, h \hookrightarrow E', h'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P, h'} \quad \text{SPAWN} \frac{E \hookrightarrow_{\text{when } (\bar{\kappa}') \{E''\}} E'}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R \uplus (\bar{\kappa}, E'), P : (\bar{\kappa}', E''), h} \\ \\ \text{RUN} \frac{(\bigcup_{(\bar{\kappa}', _) \in (P' \cup R)} \bar{\kappa}') \cap \bar{\kappa} = \emptyset}{R, P' : (\bar{\kappa}, E) : P'', h \rightsquigarrow R \uplus (\bar{\kappa}, E), P' : P'', h} \quad \text{END} \frac{\text{finished}(E)}{R \uplus (\bar{\kappa}, E), P, h \rightsquigarrow R, P, h} \end{array}$$